Computational Complexity and Intractability

Theory of NP Completeness

1

NP Completeness

Topics

Basics

- *-Intractability*
- *-Polynomial-Time Algorithms*
- *-Superpoynomial-Time Algorithms*
- *-Optimization vs Decision Problems*

Verifiability

- *-Polynomial Verifiability*
- *-Verifiability Algorithm*

Reducibility

- *-Basics of Reducibility*
- *-Reduction of Sample Problems*

Proving NP-Completeness

- *-Theorems*
- *-Reduction Paths*

2

NP Completeness

Intractability

Intractable is defined as difficult to treat or work. This means that a problem in computer science is intractable if the computer has difficulty in solving it.

There are three general categories of problems as far as intractability is concerned.

1. Problems for which ***polynomial time algorithms have been found***.
2. Problems that have ***proven to be intractable***.
3. Problems that have ***not been proven*** to be intractable, but for which polynomial time ***algorithms have never been found***.

3

NP Completeness

Polynomial Algorithms

An algorithm which has ***worst running time of $O(n^k)$***, where k is constant, for an input of size n, is called ***polynomial-time algorithm***.

The algorithms which have ***logarithmic run time*** are also classified as polynomial-time algorithm because all logarithmic running time are some polynomial upper bound.
For example, $lg\ n=O(n)$, $nlg\ n=O(n^2)$

Examples of polynomial-time algorithms are
$\sqrt{n}$, $n^{1.5}$, $n+5n^2$, n^{10}, $nlg\ n$, $lg\ n$, $n^2+nlg\ n$, $n(lg\ n)^3$

The polynomial algorithms are considered ***efficient***. For such reason, the problems which can be solved by polynomial-time algorithms are classified as ***tractable*** or ***easy***

4

NP Completeness

Polynomial Algorithms

An algorithm which has ***worst running time of $O(n^k)$,*** where k is constant, for an input of size n, is called ***polynomial-time algorithm***.

The algorithms which have ***logarithmic run time*** are also classified as polynomial-time algorithm because all logarithmic running time are some polynomial upper bound.
For example, $lg\ n=O(n)$, $nlg\ n=O(n^2)$

Examples of polynomial-time algorithms are
$\sqrt{n}$, $n^{1.5}$, $n+5n^2$, n^{10}, $nlg\ n$, $lg\ n$, $n^2+nlg\ n$, $n(lg\ n)^3$

The polynomial algorithms are considered ***efficient***. For such reason, the problems which can be solved by polynomial-time algorithms are classified as ***tractable*** or ***easy***

5

NP Completeness

Superpolynomial Algorithms

An algorithm which does not run in polynomial time for any input is called ***superpolynomial-time algorithm***.

Examples of ***superpolynomial-time*** algorithms are

2^n , n^2+3^n, $n!$, n^n , n^{kn} a^n

The superpolynomial algorithms are considered ***inefficient***. The problems which are solved by a superpolynomial-time algorithms are classified as ***intractable*** or ***hard***

6

Comparison of Algorithms

Polynomial and Superpolynomial

Assuming that CPU executes one step of algorithm in ***10^{-12} seconds***, the sizes of a problem that can handled by polynomial and superpolynomial algorithms in different time spans are tabulated below.

Duration	n^3	2^n	n!	n^{10}
1hour	$1.5x10^3$	51	17	13
10 hours (1/2 days)	$3.3x10^5$	54	18	14
100 hours (4 days)	$7.1x10^5$	58	19	14
1000 hours (1 month)	$1.5x10^6$	61	20	15
10,000 hours (1 year)	$3.3x10^6$	64	20	16
100,000 hours (1 decade)	$7.1x10^6$	68	21	16

7

NP Completeness

Polynomial Algorithms Examples

Most of the problems discussed and analyzed in sections have polynomial run times.

- *Binary search, O(lg n)*
- *Randomized search ,O(√n)*
- *Sequential Search, tree and graph traversals, O(n)*
- *Advanced sorting (quick sort, heap sort, merge sort), O(nlgn)*
- *Searching Binary Search Trees and Red-Black trees, O(n lg n)*
- *Minimum spanning tree (Kruskal's algorithm), O(|E| lg |E|), $E=n^2$*
- *Elementary Sorting (Selection sort, Insertion sort), $O(n^2)$*
- *All pairs shortest distances, $O(n^3)$*
- *Pattern matching, $O(n^3)$*

8

NP Completeness

Hard Problems

There is large number of ***intractable*** problems in computer science, mathematics, logic, and science. Despite extensive research studies over the past several decades, no efficient algorithms have been found to solve these problems. It is widely believed by computer scientists that ***no efficient algorithm exits*** for the solution of such problems Some typical examples of hard problems are:

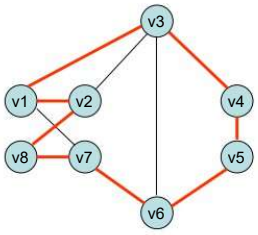
- *Hamiltonian Circuit Problem*
- *Traveling Salesperson Problem*
- *Graph Clique Problem*
- *Graph Independent Set Problem*
- *Graph Vertex Cover Problem*
- *Graph Colorability Problem*
- *Satisfiability (SAT) Problem*
- *3-CNF SAT Problem*

9

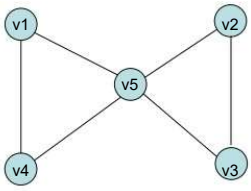
Hamiltonian Circuit Problem

A ***Hamiltonian circuit*** , also called ***tour***, is path in a graph that starts and ends at the same vertex, and passes through all other vertices exactly once. Figure *(a)* shows an example of Hamiltonian circuit

A graph can have more than one Hamiltonian circuit or ***none*** at all. Figure *(b)* shows a graph which has no Hamiltonian circuit.



(a) Hamiltonian circuit {v1,v2,v8,v7,v6,v5,v4,v3,v1}



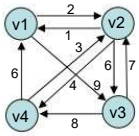
(b) Graph with no Hamiltonian circuit

Problem: *Given a Graph G=(V,E), (1)determine if the graph has Hamiltonian Circuit (2) Determine all Hamiltonian circuits*

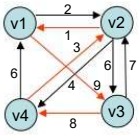
10

Traveling Salesperson Problem

A salesperson has to travel to ***n cities,*** such he starts at one city and ends up at the same city, visits each city ***exactly once*** , and covers ***minimum distance***. The problem is to determine ***minimum tour*** Figures show a sample directed graph and optimum tour.



(a) Directed graph



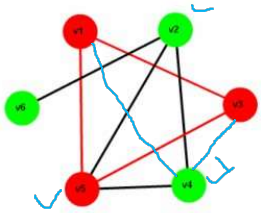
(a) Optimum tour

Problem: *Given a directed graph G=(V,E), determine optimum tour*

11

Clique Problem

. A subset of vertices *U* is a clique in graph *G=(V,E)* if there exists an edge between all pairs of vertices in *U*. In other words, clique is a complete subgraph. Size of maximum clique is called ***clique number*** denoted by ω(G) . Figure show a sample graph with a clique of size 3.



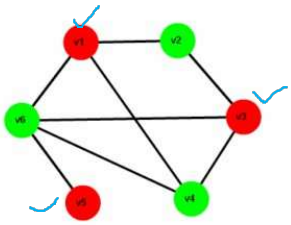
Clique U= v1,v3,v5)

Problem: Given a graph *G=(V,H)*, find a clique of maximum size

12

Independent Set Problem

. A subset of vertices *U* is an ***independent*** in graph *G=(V,E)* if there is no edge incident on any pair of vertices in *U*. The size of ***maximum independent set*** is called ***independent number*** denoted by α(G) . Figure show a sample graph with a independent set of size 3.



Independent Set U=(v1,v3,v5), the vertices have no edges between them

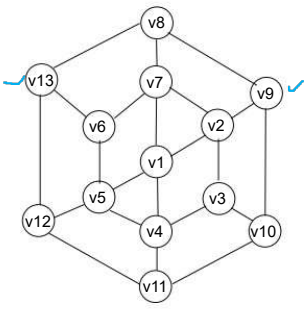
Problem: *Given a graph G=(V,E), find an independent set of maximum size*

13

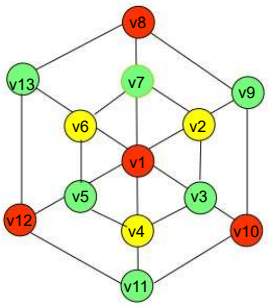
Graph Coloring Problem

. For given graph, find the ***minimum number of colors*** that can be used to color the vertices so that ***no two adjacent vertices have the same color***. The number is called ***chromatic number,*** denoted by ***χ(G)***

Figure *(a)* shows a graph consisting of 13 vertices. Figure *(b)* shows a colored version of the graph in which a ***minimum 3 colors*** are used such that no two adjacent vertices have the same color



(a) Uncolored Graph



(b) The 3- color solution

Problem: *Given a graph G=(V,E) determine if G is k-colorable, where k<|V|*

14

CNF-Satisfiability Problem

•A ***logical (Boolean) variable*** is a variable that can be assigned either 0 (false) or 1(true)
•The ***negation*** of logical variable x is $\bar{x}$. It is also called ***complement*** of x
•A ***literal*** is a logical variable or complement of a variable. For example, x and $\bar{x}$ are considered as literals
•A ***clause*** is a sequence of literals which are connected by the ***logical OR-operator***, denoted by v
•A logical expression (formula) is said to be in ***conjunctive normal form (CNF)*** if it consists of clauses connected by ***logical AND-operator***, denoted by & .
The following is an example of logical expression in CNF, which consists of five clauses:

$$\Phi=(x1 \vee \bar{x2}) \ \& \ (x2 \vee x3 \vee x4) \ \& (x1 \vee \bar{x2} \vee x4 \vee \bar{x3}) \ \& \ (x2 \vee \bar{x3} \vee x4) \ \& \ (x3 \vee x4)$$

An expression in CNF ***evaluates to 0 or 1***, depending on the values assigned to literals If the expression evaluates to ***1(true)*** for some set of assignments, the expression is said to be ***satisfiable***; If no assignment makes the expression true , the expression is called ***unsatisfiable***.

• For example, the expression
$(x1 \vee x2) \ \& \ (x2 \vee \bar{x3}) \ \& \ \bar{x2}$
is satisfiable for assignment **x1 = 1, x2 = 0, x3 = 0**

• The expression $(x1 \vee x2) \ \& \ \bar{x1} \ \& \ \bar{x2}$ is unsatisfiable; no assignment can make the expression equal to 1

Problem: *Given a logical expression (formula) φ in CNF consisting of n literals, determine if φ is satisfiable. The problem is also briefly referred to as* ***CNF-SAT*** *problem*

15

3-CNF Problem

If every clause in a CNF formula consists of ***exactly three literals***, then the expression is said to be in ***3-CNF*** . Apparently, 3-CNF is simpler compared to a general CNF expression. This an example of formula in 3-CNF, consisting of three clauses

$$\Phi = (x1 \vee \bar{x2} \vee \bar{x3}) \ \& \ (x1 \vee \bar{x2} \vee x3) \ \& \ (x1 \vee x2 \vee x3)$$

Problem: *Given a logical expression (formula) φ in 3-CNF consisting of n literals, determine if φ is satisfiable. The problem is also briefly referred to as* ***3-CNF-SAT*** *problem or simply 3-SAT*

16

Comparison of Algorithms

A comparison of tractable and intractable problems and the algorithms is given below

Problem	Algorithm
Binary Search	$lg\ n$
Randomized Search	$\sqrt{n}$
Sequential Search, Tree/Graph traversals	n
Advanced Sorting (Quick, Heap, Merge) , BST search	$n\ lg\ n$
Elementary Sorting	n^2
Shortest paths	n^3
Cliques, vertex cover, SAT, graph colorability	2^n
Hamiltonian circuit, Traveling Sales Person	$n!$

efficient

inefficient

17

NP Completeness

Decision *versus* Optimization Problems

Most problems that are encountered in practice are classified into two major types Decision problems and Optimization problems.

Decision Problems: The decision problems are problem whose solution can be expressed in either 'yes' or 'no' form of output . The corresponding inputs are sometimes referred to as 'positive instance' and 'negative instance' respectively. For example, if an algorithm searches for a given key k in a data set, The result could be other successful with out 'yes' (positive instance of key) or unsuccessful with output 'no'(negative instance of key).

Optimization Problems: The optimization problems require a solution which can be expressed in terms of maximum or minimum value of some objective function. Some examples are traveling sales person problem, All-pairs shortest distances in graph, minimum spanning tree, longest common subsequence

Every optimization problem can be formulated into an equivalent decision problem. Therefore, the theory of NP-completeness focuses mainly on decision problems.

18

NP Completeness

Decision and Optimization Equivalence

Some typical examples of optimization problems and their equivalent decision problems are:

Traveling Salesperson Problem:
(a) Optimization: *Given a weighted graph, find minimum tour*
(b) Decision:*Is there a a tour of total distance less than total weight w ?*

Minimum Spanning Tree:
(a) Optimization: *Given an undirected weighted graph find minimum spanning tree*
(b) Decision: *Is there a spanning tree with total weight less than d ?*

Graph Colorability:
(a) Optimization: *Given a graph, find minimum number of colors so that no neighboring vertices have the same color.*
(b) Decision: *Is graph k-colorable, where k<|V\ ?*

Clique Problem:
(a) Optimization: *Given a graph find a clique of maximum size.*
(b) Decision: *Does the graph have a clique of size k ?*

19

Verifiability

20

NP Completeness

Polynomial-time Verifiability

Although it may be difficult to obtain a solution of a problem , it is in general, easier to ***verify*** whether a given set of parameters (values) provide correct solution to a problem

For example, it is ***difficult*** to find a solution to the cubic equation

$$x^3 - 8x^2 + 17x - 10 = 0$$

it easy to verify that $x=1, x=2, x=5$ are ***correct solutions*** but $x=4$ is ***not a solution***.
This can be done by simply plugging the given vale of x into the equation and checking if the expression on the left side reduces to zero.

The underlying procedure for checking whether a given input is a solution is called ***verification algorithm***

While it may or may not be possible to obtain the solution in polynomial time, in many cases it is feasible to verify the solution by using a polynomial-time algorithm, called ***polynomial verification algorithm*** For example, we can use polynomial algorithms to verify, solution of a sorting problem and Hamiltonian Circuit problem.

21

Sorting Problem Verifiability

We can verify whether a given set of input is sorted by checking the consecutive input keys. If the keys are out of order the output of the algorithm would be ***false (NO)***
The pseudo code for verification is as follows. It checks if the input array A is in ascending order. If this is not the case ***false*** is returned..

```
SORT-VERIFICATION (A, n)    ► A is input array, n is number of keys
 for j←1 to n-1 do
   if A[j+1] not greater than A[j]
     then return false
return true
```

The algorithm consists of one loop which is executed $n-1$ times. Thus, running time is $O(n)$. In other words, the verification algorithm is ***polynomial***

22

Hamiltonian Circuit Verifiability

Given a graph *G* consisting of *n* vertices, the following verification algorithm can be used to certify whether or not a set of vertices U constitutes a Hamiltonian Circuit. It is assumed that graph is represented by an ***adjacency matrix***

*(1)Check if the **first vertex** and **last vertex** in U are linked*
*(2)Check that total number of **vertices in U equals n***
*(3)Check if all pairs of **consecutive vertices** are linked together*

The pseudo code for the verification algorithm is as follows:

```
HAMILTONIAN-VERIFICATION (A, U, n)   ► A is Adjacency Matrix, U is vertices array
if length[U ] ≠ n then
    return false
if A[U[1],U[n]] ≠ 1 then
    return false
for j←1 to n-1 do
    if A[U[j+1], U[j]} ≠ 1 then
        return false
return true
```

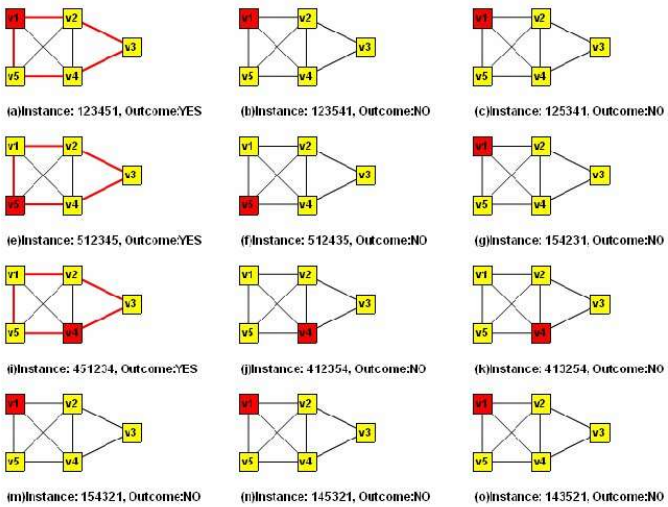
The algorithm consists of one main loop which is executed *n-1* times. Thus, running time is *O(n)*. Observe that whereas the solution to Hamiltonian problem has ***superpolynoial*** run time , the verification algorithm runs in ***polynomial*** time

23

Hamiltonian Circuit Verification

Example

The figure shows an example of Hamiltonian Circuit Verification for different inputs. The Hamiltonian Circuit is shown for YES instances. Input consists of vertices coded as integers. For example input 125341 represents the path {v1,v2,v5,v3,v4,v1}



24

NP Completeness

P Class

A decision problem is said to belong to ***P Class*** if it can be solved by ***polynomial-time algorithm***. As discussed earlier, these problems are ***tractable***

The following are some examples: of problems in class P

- *Minimum spanning Tree*
- *All-Pairs Shortest Distances*
- *Searching Problems*
- *Sorting Problems*

25

NP Completeness

NP Class

A decision problem is said to belong to ***NP Class*** if

*it can be solved by **polynomial-time** or **superpolynomial-time** algorithm.*

*any guessed or proposed solution can be **verified** by **polynomial-time** algorithm*

The following are some examples of NP problems:

- *Sorting and searching problems*
- *Hamiltonian circuit problem*
- *Traveling Salesperson problem*
- *Graph Colorability*

26

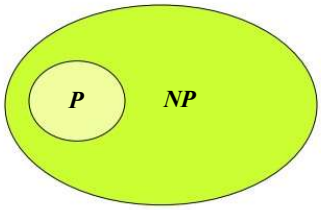
NP Completeness

P and NP Classes

Since problems in class P can be ***solved*** and ***verified*** by ***polynomial algorithms***, the class P is in fact subset of class NP. Symbolically, the relationship can be expressed as:

$$P \subseteq NP$$

which is shown pictorially in the figure



27

NP Completeness

NP-Complete Class

A decision problem is said to belong to ***NP-Complete Class*** if
(1) it is hard that is ***superpolynomial-time*** *algorithm provides solution.*
(2) any guessed or proposed solution can be ***verified*** *by* ***polynomial-time*** *algorithm*

The following are some examples of NP-Complete problems:

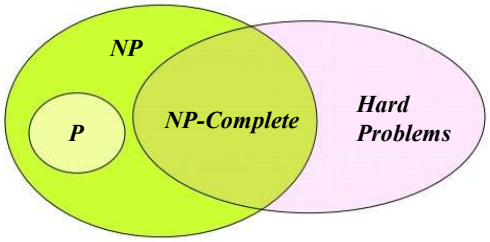
- *Is there a Hamiltonian circuit in the graph ?*
- *Is there a tour of total length less than d ?*
- *Does given graph have a clique of size k ?*
- *Does given graph has a vertex cover of size k ?*
- *Is given graph k-colorable ?*
- *Is a given logical expression in CNF satisfiable ?*

28

NP Completeness

NP-Complete and Hard Problem Classes

The figure shows the relation between NP-Complete Class and Hard problems Recall that ***hard*** or ***intractable*** problems have superpolynomial run time



The following problems are hard but do not belong to NP-Class:

(1) Are there more than k Hamiltonian circuits in a given graph ?
(2) Are there more than k Traveling Salesperson tours of total length d ?

In both cases, the count ***cannot be verified by polynomial-time algorithm***, without first finding all the paths

29

Proving NP-Completeness

30

NP Completeness

Problem Reduction

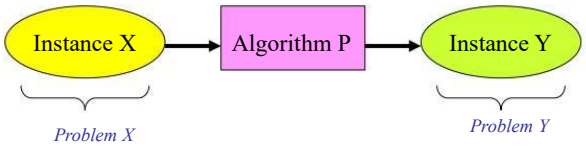
The NP-Complete problems have a remarkable property that a given problem X can be translated into another NP-Complete problem Y, such that the transformation converts Any positive instance (YES) of X into a positive instance of Y and a negative instance (NO) of X into negative instance of Y. Further, the transforming algorithm runs in polynomial time.

The problem reduction procedure is symbolically denoted by the notation

$$X \leq_P Y$$

where suffix p indicates that transformation is polynomial

The translation steps are illustrated in the figure. The input to the algorithm P is an an instance of problem X, and output is an instance of Y.



31

NP Completeness

Reduction Examples

Depending upon the problem, the reduction procedure may be simple or quite complex. Here are some typical examples of transformation

- *Clique problem to Independent set problem*
- *Independent Set problem to Vertex Cover problem*
- *Clique problem to graph coloring problem*
- *SAT problem to 3-CNF SAT problem*
- *3-CNF SAT problem to clique problem*
- *3-CNF SAT to Independent Set problem*
- *Vertex Cover problem to Hamiltonian problem*
- *Hamiltonian problem to Traveling Salesperson problem*

32

Reduction of Clique to Independent Set

Procedure

Recall that a ***Clique*** is a subgraph *U* of graph *G* such *U* is complete graph i.e every vertex in *U* is linked to all other vertices. An ***Independent Set*** is a subgraph *I*, such that no links exist between any pair of the vertices in *I*

The procedure for transformation of Clique to is simple. If *G=(V,E)* is a given graph with Clique of size *k*, the ***complement*** *G'* of *G* contains an Independent Set of size *k*. A complement G' of a graph G is such that if an edge (u,v) exists in G then it does not exist in G' and vice versa. In other words, the graph G' is obtained by the following procedure:

(1) Remove all existing edges

(2) Add all missing edge

The ***resulting graph G' will be the complement of G***. If G contains clique of k vertices, the Graph G' would contain an Independent Set consisting of the corresponding vertices of clique

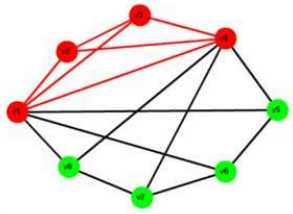
The above procedure can be implement in polynomial run time by using an Adjacency Matrix A which represents G. An entry '1' in the matrix indicates a link between a pair of vertices and entry '0' indicates a missing edge. Thus, by ***flipping the non-diagonal entries in A***, the adjacency matrix for complement G' is obtained. The flipping involved at most n^2 steps for graph with n vertices. Thus, ***transformation algorithm runs in polynomial time O(n^2)***

33

Reduction of Clique to Independent Set

Example

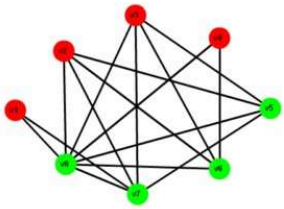
The procedure for transformation of a Clique to an Independent Set is illustrated in the figures. Figure (a) shows a sample graph G together with the adjacency matrix. It has cliques of size 4. Figure (c) shows ***complement*** graph G' and the associated adjacency matrix. The graph G' has an Independent Set of four vertices



(a) G with clique {v1,v2,v3,v4}

0	1	1	1	1	1	0	1
1	0	1	1	0	0	0	0
1	1	0	1	0	0	0	0
1	1	1	0	1	0	1	1
1	0	0	1	0	1	0	0
1	0	0	0	1	0	1	0
0	0	0	1	0	1	0	1
1	0	0	1	0	0	1	0

(b) Adjacency matrix of G



(c) G' with Independent Set {v1,v2,v3,v4}

0	0	0	0	0	0	1	0
0	0	0	0	1	1	1	1
0	0	0	0	1	1	1	1
0	0	0	0	0	1	0	0
0	1	1	0	0	0	1	1
0	1	1	1	0	0	0	1
1	1	1	0	1	0	0	0
0	1	1	0	1	1	0	0

(d) Adjacency matrix of G'

34

Proving NP Completeness

Theorems

The following theorem are used to prove the NP-Completeness of problems

Cook-Levin Theorem: *(1) SAT Problem is a NP-Complete*
(2) 3-CNF Problem is NP-Complete

Reduction Theorem: If X and Y are problems in NP, X is NP-Complete, and $X \leq_P Y$ then Y is NP-Complete

The Cook-Levin theorem was propounded independently by ***Stephen Cook*** and ***Lenoid Levin*** in 1971. It was a ***ground breaking theorem*** that set the ball rolling to prove the NP-Completeness of a variety of problems. The proofs for NP-Completeness of SAT and 3-CNF SAT are based on mathematical logic. The proofs are quite complicated

The second theorem provides the standard procedure for proving NP-Completeness of given Problem. If we know some known NP-Complete problem and by using polynomial Reduction algorithm we can transform it to given problem, then according to Reduction Theorem The given problem would also be NP-Complete. For example, according to Cook-Levin Theorem, 3-CNF SAT problem is in NP-Complete. If we can reduce 3-CNF SAT problem to Clique problem by polynomial reduction, then Clique problem would be NP-Complete.

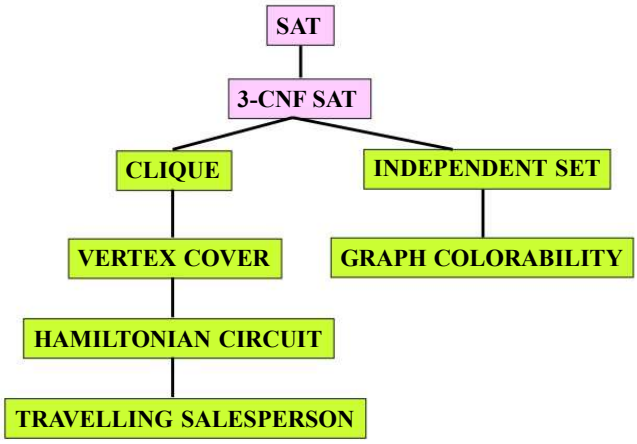
In general, the procedure for proving NP Completeness boils down to a sequence of polynomial reductions.

35

NP Completeness

Proof by Reduction

The NP Completeness of several problems can be proved by sequence of ***Polynomial Reductions.*** The path for reduction for some the typical problems is shown in the figure.



While procedure for conversion of Clique, Vertex Cover, Graph Colorablity is simple, it is quite complicated for reduction of Vertex Cover to Hamiltonian Problem

36